Dr. Vishwanath Karad MIT World Peace University

Department of Computer Engineering and Technology

Mini Project using Java Programming – Laboratory Manual

Academic Year: 2025–26 (Even Semester)

Program: TY B.Tech – Computer Science & Engineering

Course Code: CSE3PR02A

Course Title: Mini Project using Java Programming

Credits: 01

Contact Hours: 03 Lab Hours / Week

Course Coordinator: Dr. Swati Jadhav

Course Teachers: Dr. Sheetal Girase, Dr. Ranjana Agrawal, Dr. Vittal Gutte, Santushti Betgeri

STUDY ASSIGNMENT

Title: Study of Java Programming Environment

Aim

To study Java SDK, J2EE, JVM architecture, and Java IDEs.

Objectives

- Understand Java SDK components
- Study JVM architecture and execution flow
- Learn the role of J2EE
- Familiarize with Eclipse and NetBeans IDE

Theory

Java SDK includes compiler, interpreter, and development tools.
JVM converts bytecode into machine-specific instructions.
J2EE provides technologies for enterprise-level web applications.
IDEs simplify Java development using debugging and project tools.

Platform

Eclipse IDE / NetBeans IDE

Conclusion

The Java programming environment and JVM architecture were studied.

FAQs

1. What is Java SDK?
2. What is the role of JVM?
3. What is bytecode?
4. How does Java achieve platform independence?
5. What is J2EE?
6. Difference between JDK and JRE?
7. Name popular Java IDEs.

EXPERIMENT 1

Title: Basic Java Program

Aim

To write a Java program to print
“Welcome to Java Programming World”

Theory

A Java program starts execution from the main() method.
The System.out.println() statement is used to display output.

Sample Program (Skeleton)

```
public class Welcome {
   public static void main(String[] args) {
      System.out.println("Welcome to Java Programming World");
   }
}
```

Conclusion

A basic Java program was executed successfully.

FAQs

1. What is the main() method?
2. Why is main() declared static?
3. What is public access specifier?
4. What is the use of System.out?
5. What happens if main() is not present?

EXPERIMENT 2

Title: Classes, Objects, Methods and Constructors

Aim

To demonstrate classes, objects, methods, constructors, and access specifiers using student information.

Theory

A class defines properties and behavior.
Objects are instances of a class.
Constructors initialize object state.
Access specifiers control data visibility.

Sample Skeleton Code

```
class Student {
   private int rollNo;
   private String name;

   Student(int rollNo, String name) {
      this.rollNo = rollNo;
      this.name = name;
   }

   void display() {
      System.out.println(rollNo + " " + name);
   }
}
```

Conclusion

OOP concepts were implemented using Java.

FAQs

1. What is a class?
2. What is an object?
3. What is a constructor?
4. What is encapsulation?
5. What is the default access specifier?
6. Difference between public and private?

7. Can a constructor return a value?

EXPERIMENT 3

Title: Inheritance, Abstract Class, Interface and File I/O

Aim

To demonstrate inheritance and implement abstract class, interface, and file handling.

Theory

Inheritance allows one class to reuse another class's properties.
Abstract classes provide partial implementation.
Interfaces define behavior contracts.
File I/O enables data persistence.

Conclusion

Inheritance and file handling concepts were implemented.

FAQs

1. What is inheritance?
2. Types of inheritance in Java?
3. Difference between abstract class and interface?
4. Can an interface have methods with body?
5. What are I/O streams?
6. Difference between FileInputStream and FileReader?

EXPERIMENT 4

Title: Method Overloading and Method Overriding

Aim

To implement compile-time and runtime polymorphism.

Theory

Method overloading occurs at compile time.
Method overriding occurs at runtime and supports dynamic binding.

Conclusion

Polymorphism was implemented successfully.

FAQs

1. What is polymorphism?
2. What is method overloading?
3. What is method overriding?
4. Can static methods be overridden?
5. Difference between early and late binding?

EXPERIMENT 5

Title: Multithreading and Exception Handling

Aim

To implement multithreading using Thread and Runnable and handle exceptions.

Theory

Multithreading enables concurrent execution.
Exception handling prevents abnormal program termination.

Sample Skeleton Code

```
class MyThread extends Thread {
   public void run() {
      System.out.println("Thread executing");
   }
}
```

Conclusion

Concurrent execution and exception handling were studied.

FAQs

1. What is multithreading?
2. Difference between Thread and Runnable?
3. What is synchronization?
4. What is an exception?
5. Difference between checked and unchecked exceptions?
6. What is finally block?

EXPERIMENT 6

Title: Web Application using Servlets, JSP and JDBC

Aim

To develop a simple Java web application.

Theory

Servlets process client requests.
JSP generates dynamic web content.
JDBC connects Java with databases.

Conclusion

A simple web application was developed.

FAQs

1. What is a servlet?
2. What is JSP?
3. Servlet lifecycle methods?
4. What is JDBC?
5. What is MVC architecture?
6. Difference between forward and redirect?